

이 미 지 · 센 서 · 화 면 반 영

우수주의보

이미지·센서 기반 빗물받이 상태 판단 및 관리 화면 반영

YOLO OpenCV XGBoost PostgreSQL WebSocket

발표 핵심

2026

이 미 지



센 서 값



화 면 반 영



한 문장 요약

sample image와 mock sensor data를 분석 작업으로 연결하고, 결과 저장 후 화면 갱신까지 이어지는 흐름을 구현했습니다.

프로젝트 한 줄 요약

입력, 분석, 저장, 반영을 하나의 요청 흐름으로 연결해 개별 빗물받이 상태를 화면에서 확인하게 합니다.



팀 구성과 역할

개인 소개보다 발표 흐름에 필요한 책임 단위와 산출물을 먼저 보여줍니다.

AI pipeline

담당: YOLO/OpenCV 처리. 산출물: 이미지 feature. 기술: YOLO, OpenCV.

Backend / DB

담당: 작업 생성과 저장. 산출물: callback, ERD. 기술: FastAPI, PostgreSQL.

Frontend / Infra

담당: 화면 반영과 배포 구성. 산출물: dashboard/detail, Docker. 기술: React, Nginx, Jenkins.

분류 기준 / 발표

담당: feature 정의와 설명 흐름. 산출물: 판단 기준, 시연 시나리오. 기술: XGBoost.

협업 기준

회의 파트별 진행 상황과 blocker를 짧게 공유합니다.

이슈 관리 기능, 데이터, 발표 리스크를 항목별로 분리합니다.

브랜치/PR 작업 단위 변경을 기록하고 통합 시점에 충돌을 줄입니다.

발표 준비 시연 가능 범위와 과장 금지 표현을 같이 점검합니다.

목차

문제 인식에서 판단 구조, 구현 근거, 시연 가능 범위, 확장 가능성 순서로 전개합니다.

01

개발 필요성과 벤치마킹

왜 빗물받이 상태 판단 흐름이 필요한지 정리합니다.

02

AI 분석 설계

이미지 근거와 센서 근거가 최종 분류로 이어지는 구조를 설명합니다.

03

웹 서버 및 서비스 구축

비동기 분석, DB 저장, 화면 갱신의 구현 흐름을 보여줍니다.

04

인프라 / 운영 설계

배포 구성과 운영 점검 항목을 현재 구현 범위 안에서 설명합니다.

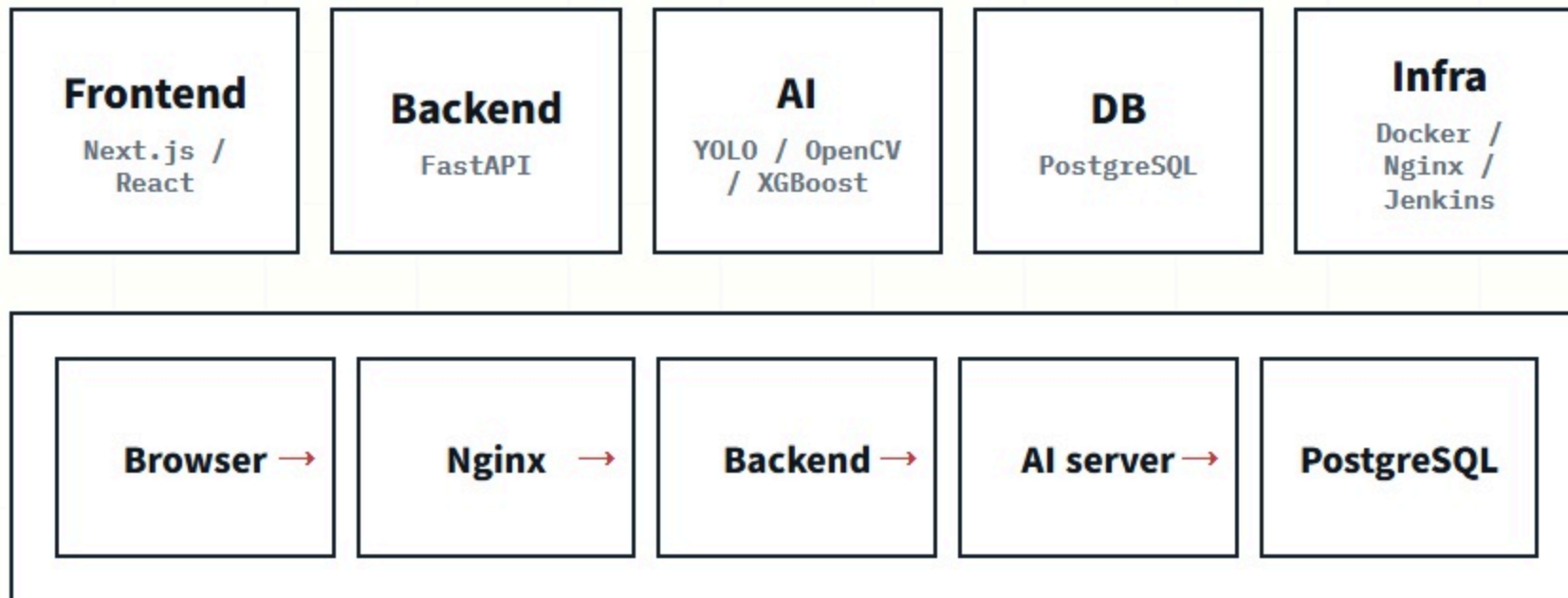
05

시연과 마무리

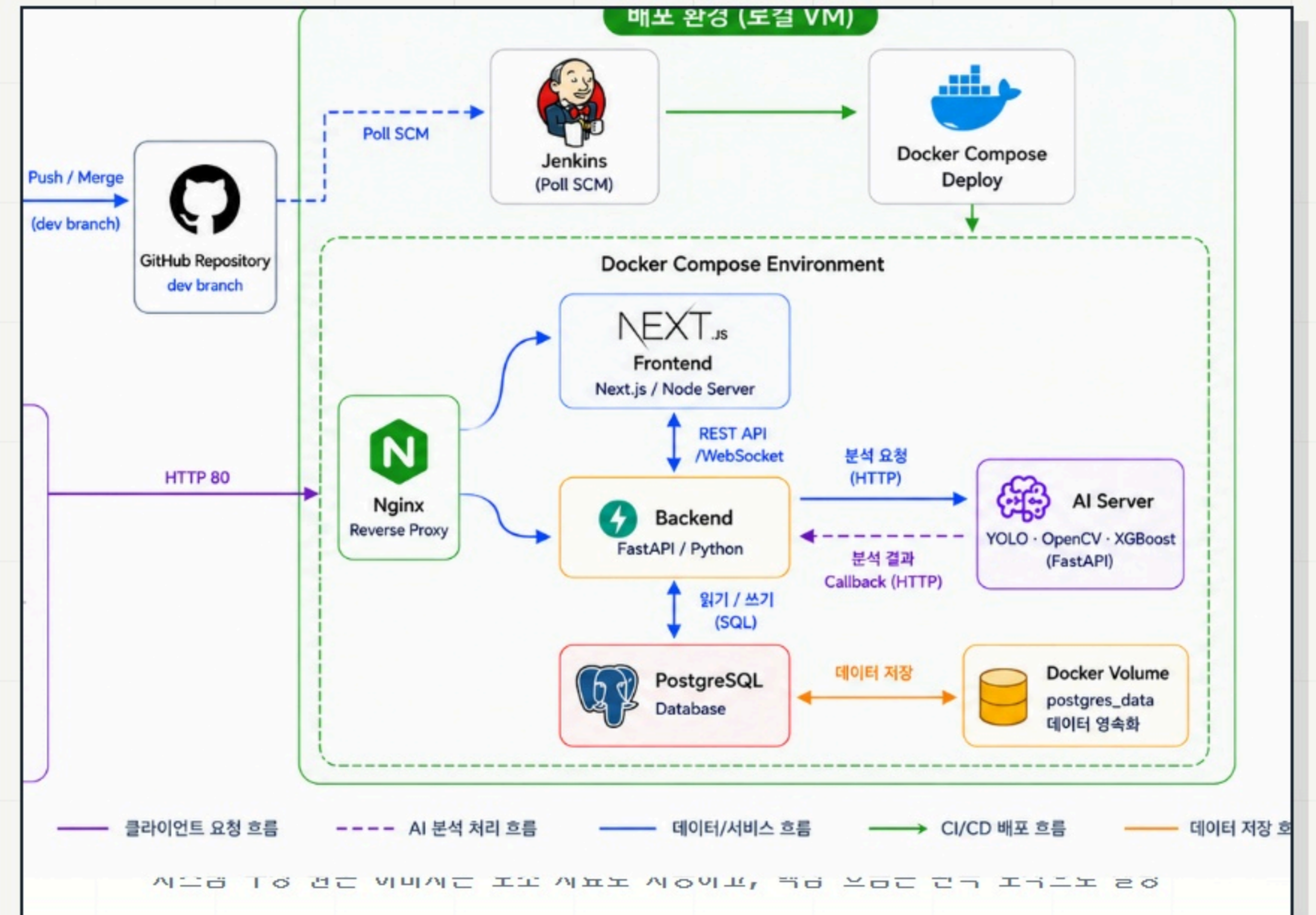
sample image와 mock sensor data 기반 시연 흐름을 연결합니다.

전체 기술 스택/구성 한 장 요약

화면, API, AI 분석, 저장소, 배포 구성이 어떤 책임으로 연결되는지 먼저 잡고 들어갑니다.



작은 글자보다 책임 흐름을 중심으로 설명합니다. 상세 아키텍처는 29번에서 다시 확대합니다.



도시 침수와 빗물받이

이 프로젝트의 목표는 도시 전체 침수 예측이 아니라 개별 빗물받이 상태를 빠르게 확인하는 흐름입니다.

왜 상태 판단이 필요한가

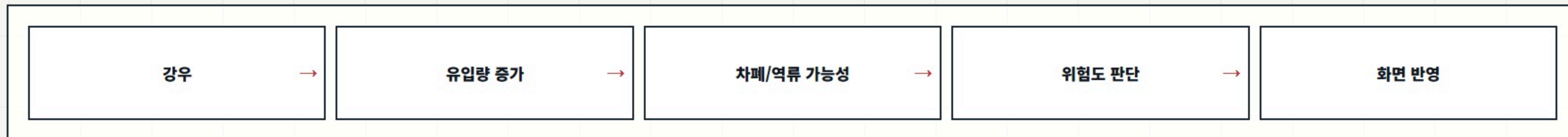
- 빗물받이는 도로 빗물이 배수관으로 들어가는 첫 지점입니다.
- 낙엽, 쓰레기, 토사로 막히면 강우 중 배수 흐름이 제한됩니다.
- 개별 시설 상태를 확인해야 점검 우선순위를 잡을 수 있습니다.



sample image: 막힘 상태를 발표에서 설명하기 위한 빗물받이 예시

강우 중 상태 변화

강우 중에는 유입량, 차폐 가능성, 수위·유속 변화가 함께 움직이므로 화면 반영까지 이어지는 판단 흐름이 필요합니다.



시각화 필요

상태가 바뀐 시설을 지도와 목록에서 빠르게 구분해야 합니다.

동시성 필요

여러 시설이 동시에 변할 수 있으므로 비동기 처리와 갱신 trigger가 필요합니다.

기존 점검 방식의 한계

기존 점검은 유지관리에 필요하지만, 강우 중 개별 시설 상태를 즉시 정렬하는 데는 공백이 있습니다.

기존 유지관리의 필요성

정기 점검과 청소는 기준, 이력, 위치 관리를 만드는 기본 활동입니다.

정기 점검

청소

이력 관리

강우 중 판단 공백

시민 신고는 현장 발견에 강점이 있지만, 신고 전 상태 변화와 우선순위를 자동으로 정렬하기 어렵습니다.

상태 변화

우선순위

화면 반영

따라서 이미지와 센서 기반 판단을 통해 우선 확인 대상을 제시하는 흐름이 필요합니다.

기존 방식/서비스 벤치마킹

벤치마킹은 디자인 참고가 아니라 상태 판단 서비스가 가져야 할 요구사항을 뽑는 과정입니다.

방식	강점	우수주의 보가 반영할 요구사항
정기 점검	기준과 이력 관리	강우 중 상태 변화가 생긴 뒤 화면에 반영되는 흐름
시민 신고	현장 발견과 민원 접수	신고 전 시설별 우선 확인 대상을 제시하는 보조 흐름
QR / 관리번호	위치 특정과 이력 연결	시설 단위 DB 추적성과 상세 화면 연결
센서 / 관제	수치 상태와 다지점 모니터링	이미지 근거와 센서 근거를 함께 쓰는 판단 구조

결론: 발견, 판단, 저장, 화면 반영을 하나의 시나리오로 묶는 것이 차별점입니다.

우수주의보 차별점

신고 대체가 아니라 이미지·센서 근거를 저장하고 화면에 반영해 점검 우선순위를 보조합니다.

이미지 근거

sample image에서 막힘 후보, 차폐물, 이미지 품질을 확인합니다.

센서 근거

mock sensor data로 수위와 유속 feature를 함께 사용합니다.

DB 추적성

작업, 결과, 상태를 PostgreSQL 기준으로 연결합니다.

UI 실시간 반영

WebSocket event 이후 대시보드와 상세 화면이 최신 상태를 조회합니다.

FLOW

근거 기반 상태 판단

기존 관리 흐름에 이미지 근거와 센서 근거를 결합하고, 내부 분류 결과를 저장한 뒤 화면 갱신 trigger로 연결합니다.

기존 관리 →

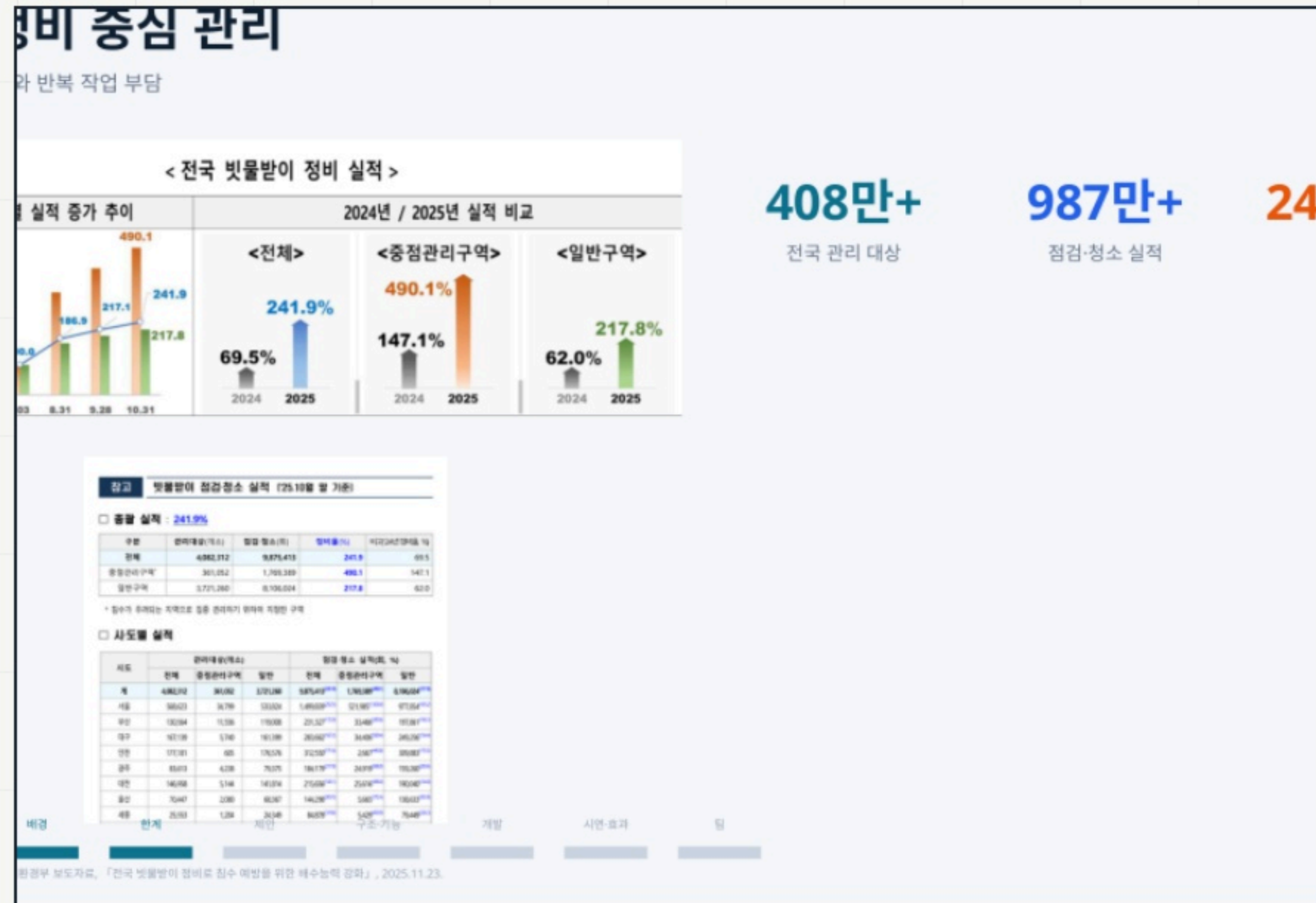
근거 결합 →

내부 분류 →

화면 반영

참고 자료/정책/사례 기반 목표

근거가 없는 비용이나 현장 검증 수치 대신, 구현 가능한 상태 판단과 저장·반영 흐름을 목표로 잡았습니다.



정량 기준

차폐율, 탐지 신뢰도, 수위, 유속을 내부 분류 feature로 정리합니다.

운영성

작업 상태, callback 결과, 실패 로그를 추적 가능한 구조로 둡니다.

화면 반영

WebSocket event 후 대시보드와 상세 화면이 최신 상태를 다시 조회합니다.

출처 표기: 제공 발표 리소스와 정책/사례 참고 페이지를 기반으로 범위를 설정했습니다.

일정표

각 단계는 다음 구현 단계의 입력물이 되도록 산출물을 남기는 방식으로 진행했습니다.

01

기획·설계

문제 정의와 서비스 흐름 정리

산출물: 발표 시나리오

02

AI 분석 설계

YOLO, OpenCV, XGBoost
기준 구성

산출물: feature 정의

03

Backend/DB 구현

작업 생성, callback, 저장
구조 구현

산출물: ERD / API 흐름

04

Frontend 구현

대시보드와 상세 화면 구성

산출물: 화면 캡처

05

통합·시연 준비

전체 흐름 검증과 발표자료
정리

산출물: demo flow

AI 분석 설계 개요

AI 분석은 이미지 근거와 mock sensor data를 feature로 만들고, XGBoost가 내부 상태 분류로 연결합니다.



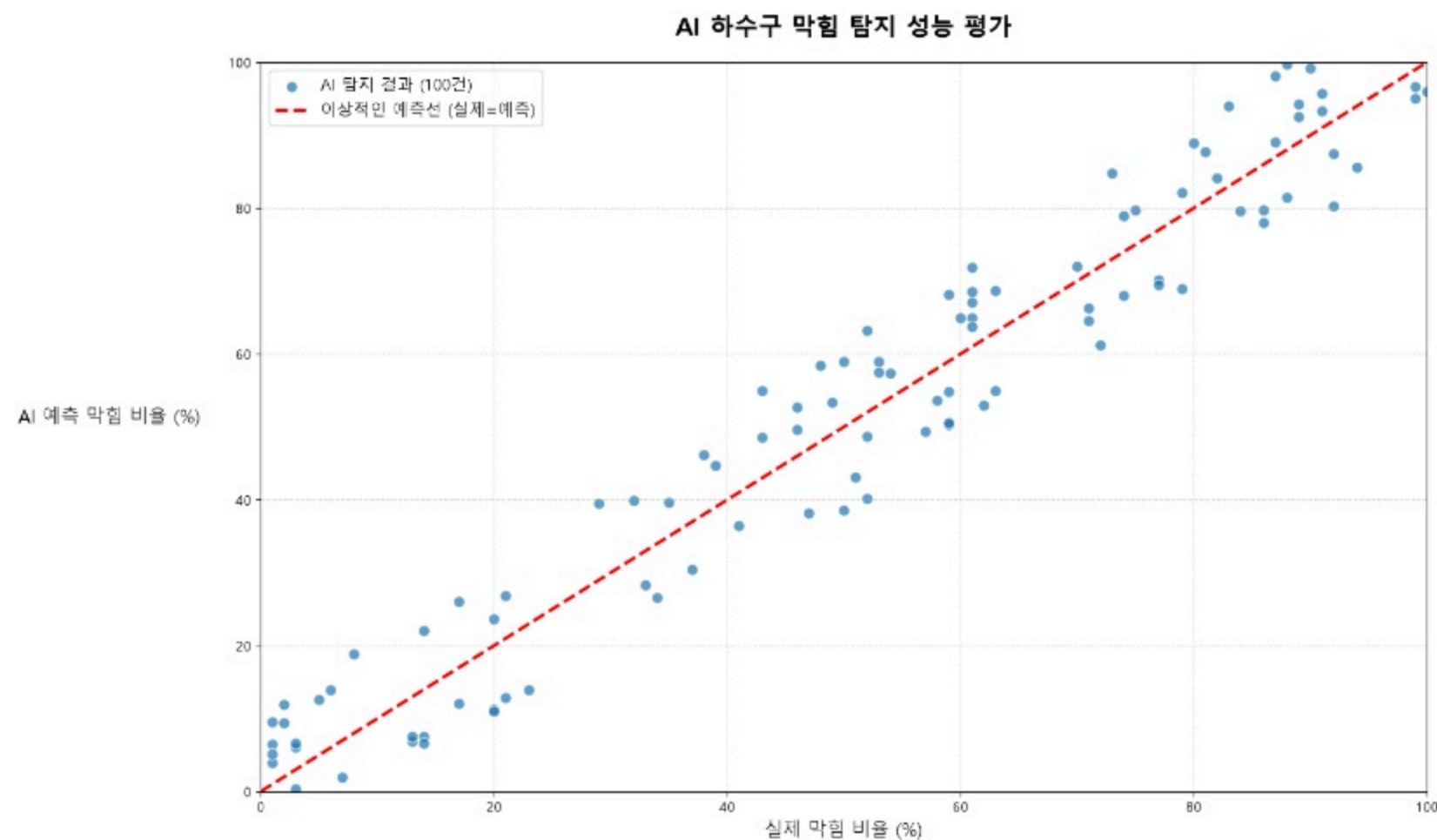
YOLO 단독이 아니라 OpenCV와 XGBoost를 결합하는 이유는 “보이는 객체”를 “판단 가능한 feature”로 바꾸기 위해서입니다.

YOLO 단독 분석 한계

객체 탐지 결과만으로 배수 위험도를 안정적으로 판단하기 어렵기 때문에 후처리와 센서 feature가 필요합니다.

발표 핵심

- YOLO는 보이는 객체와 후보 영역 탐지에 강점이 있습니다.
- 어두운 이미지, 빗물 반사, 낙엽/토사 혼재 상황은 해석을 혼듭니다.
- 따라서 mask, 품질 보정, 센서 feature가 함께 필요합니다.



보조 그래프: YOLO 단독 해석이 흔들릴 수 있어 후처리 기준을 추가한다는 인사이트를 설명

OpenCV 결합 이유

OpenCV는 탐지 결과를 실제 feature로 바꾸는 중간 단계입니다.

구 빗물받이 이미지.png - 총 막힘 비율 46.7%

2. OpenCV 전처리

The use_column_width parameter has been deprecated and will be removed in a future release. Please utilize the width parameter instead.

3. YOLO (쓰레기: 20.23%)

The use_column_width parameter has been deprecated and will be removed in a future release. Please utilize the width parameter instead.

4. OpenCV 통한 겹쳐진 탐지 (흙: 33.18%)

The use_column_width parameter has been deprecated and will be removed in a future release. Please utilize the width parameter instead.

이러한 예시: raw Image에서 mask과 어두운 영역을 복원에 obstruction ratio feature로 만들

feature로 바뀌는 지점

mask

후보 영역을 분리해 막힘 영역 계산의 기준으로 사용합니다.

dark area

어두운 이미지 조건을 품질 판단 feature로 반영합니다.

ratio

후보 영역과 차폐 영역을 obstruction ratio로 정리합니다.

차폐율·이미지 품질·객체 탐지

이미지 feature는 최종 분류를 직접 주장하는 값이 아니라, 센서 feature와 함께 판단 근거로 쓰이는 입력입니다.

차폐율

막힌 영역 비율을 설명합니다. 값이 높으면 주의·위험 판단에 영향을 줍니다.

이미지 품질

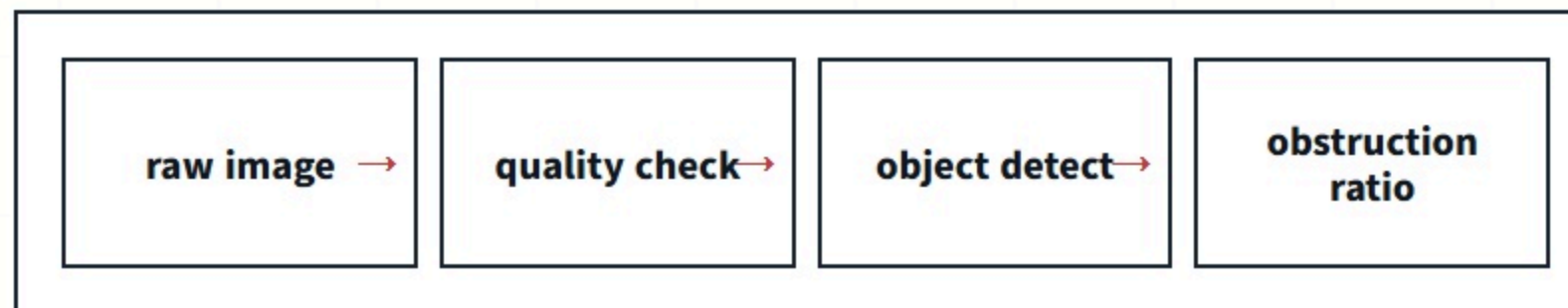
어두움, 반사, 흐림 조건을 확인해 결과 해석 신뢰도를 보정합니다.

탐지 신뢰도

객체 탐지 confidence를 함께 보고 낮은 경우 판단불가 가능성을 둡니다.

판단불가

근거 부족을 실패로 숨기지 않고 화면에서 재확인 상태로 보여줍니다.



raw image가 바로 상태가 되는 것이 아니라 품질 확인, 탐지, 영역 해석을 거쳐 feature가 됩니다.

XGBoost 입력 feature

XGBoost에는 이미지 feature와 mock sensor data feature가 함께 들어가며, 결측은 판단불가 또는 보수적 분류 근거가 됩니다.

feature	출처	의미	결측 처리 기준
obstruction ratio	OpenCV 후처리	이미지 기반 막힘 정도	낮은 품질이면 판단불가 후보
detection confidence	YOLO 결과	탐지 신뢰도	낮으면 이미지 근거 가중 제한
water level	mock sensor data	수위 상승 여부	없으면 센서 근거 부족으로 표시
flow velocity	mock sensor data	유속 저하 여부	없으면 이미지 근거 중심으로 제한

핵심은 이미지 근거와 센서 근거를 분리해 저장하고, 최종 분류에서는 함께 조합한다는 점입니다.

위험도 분류 로직

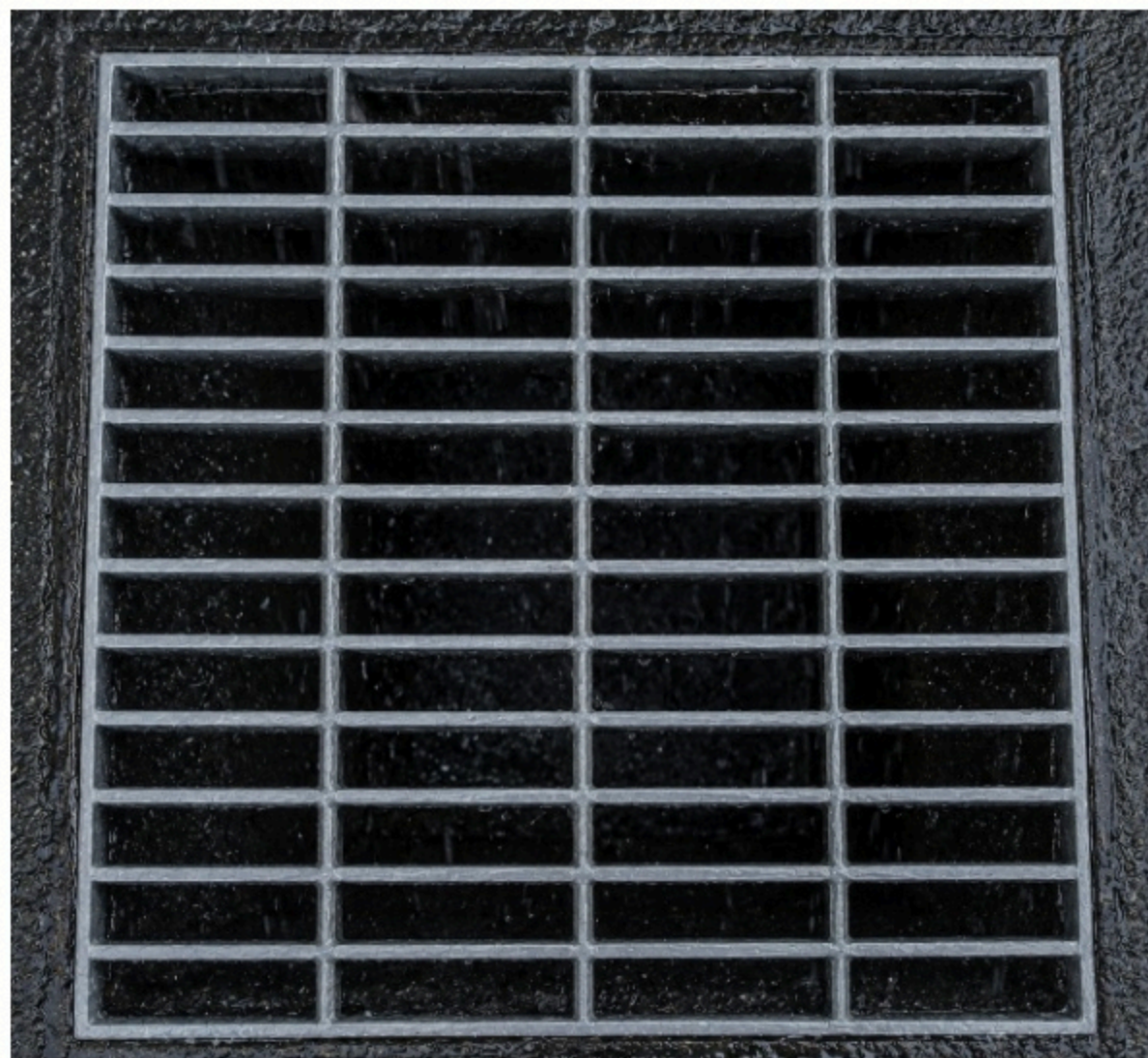
상태 분류는 점검 우선순위를 설명하기 위한 내부 계산 결과이며, 실제 물리적 침수 확률로 말하지 않습니다.

상태	판단 조건 예시	화면 처리	발표 표현
양호	차폐율과 센서 정체 징후가 낮음	일반 상태로 표시	우선 확인 낮음
주의	막힘 또는 수위·유속 변화 징후가 있음	목록에서 점검 후보 표시	관찰 필요
위험	이미지와 센서 근거가 함께 강함	대시보드 우선 확인 대상으로 표시	우선 확인
판단불가	이미지 품질 낮음, feature 결측, 분석 실패	재분석 또는 현장 확인 상태로 표시	재확인 필요

`risk\_score`는 우수주의보 내부 분류용 계산 점수이며 실제 현장 검증 기반 침수 예측값이 아닙니다.

결과/검증

검증은 sample image와 mock data 기반으로 입력, 중간 feature, 최종 분류가 연결되는지 확인하는 범위입니다.



sample image: 상태 예시와 이미지 feature 설명에 사용하는 리소스

01

입력

sample image와 mock sensor data를 하나의 분석 요청에 묶습니다.

02

중간 feature

obstruction ratio, confidence, water level, flow velocity를 확인합니다.

03

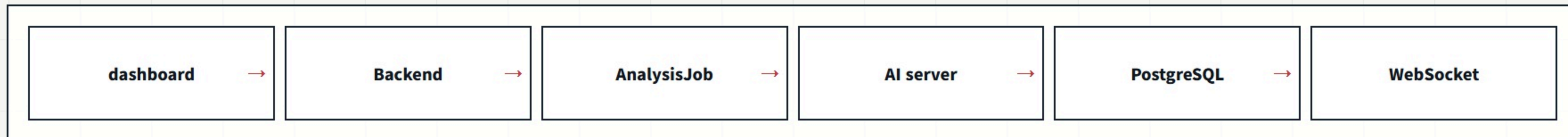
최종 분류

상태와 내부 계산 점수를 저장 가능한 결과로 만듭니다.

한계: 실제 현장 검증 완료나 실제 침수 예측 성능을 주장하지 않습니다.

서비스 구축 개요

Frontend는 분석을 직접 수행하지 않고, Backend가 AnalysisJob을 기준으로 비동기 요청과 결과 저장을 관리합니다.



AnalysisJob 상태

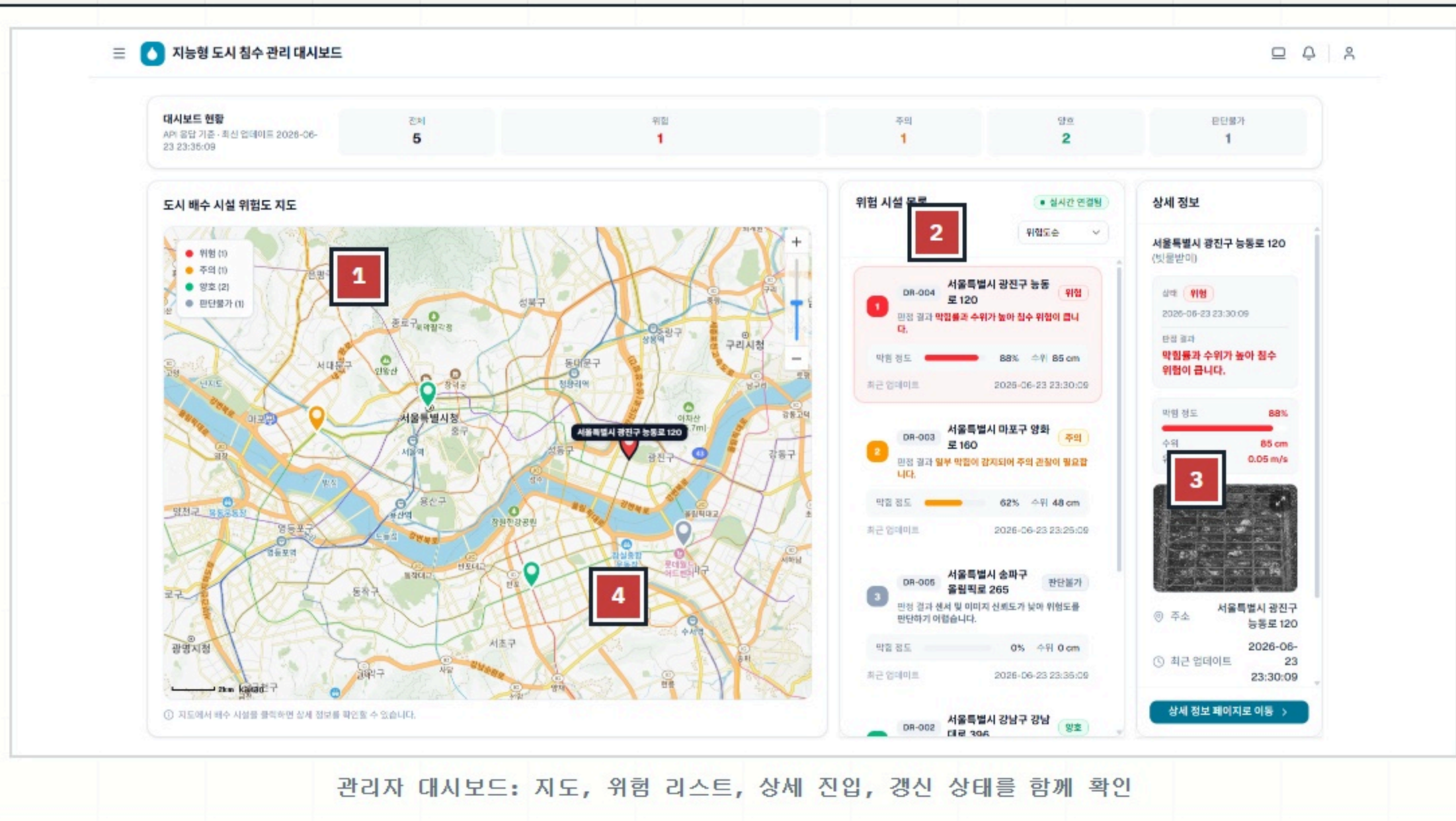
pending -> processing -> done 또는 failed로 작업 흐름을 추적합니다.

Frontend가 직접 분석하지 않는 이유

화면 응답성과 분석 처리 시간을 분리하고, 결과 저장 책임을 Backend에 고정하기 위해서입니다.

관리자 대시보드 흐름

대시보드는 전체 위치, 위험 시설, 상세 진입, 갱신 상태를 한 화면에서 확인하는 진입점입니다.



1 지도: 시설 위치와 상태를 공간적으로 확인합니다.

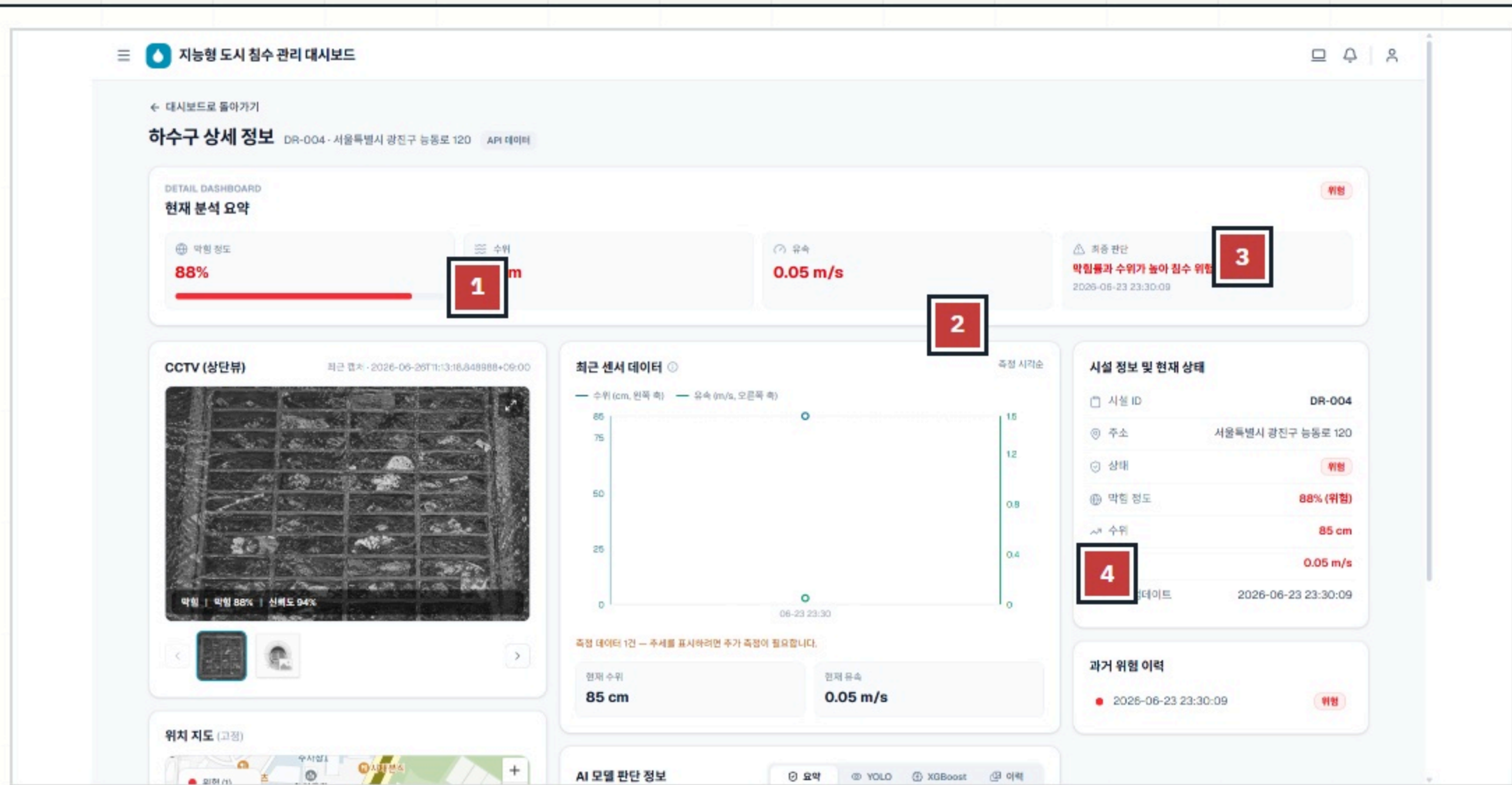
2 위험 리스트: 먼저 확인할 시설을 빠르게 찾습니다.

3 상세 진입: 선택 시설의 이미지·센서 근거로 이동합니다.

4 상태 갱신: WebSocket event 이후 최신 데이터를 다시 조회합니다.

상세 화면 흐름

상세 화면은 왜 특정 상태로 판단됐는지 이미지, 센서, 결과, 메타정보를 시설 단위로 확인하는 화면입니다.



상세 화면: 판단 근거와 저장된 결과를 시설 단위로 확인

1 이미지 분석 결과: sample image와 이미지 feature를 확인합니다.

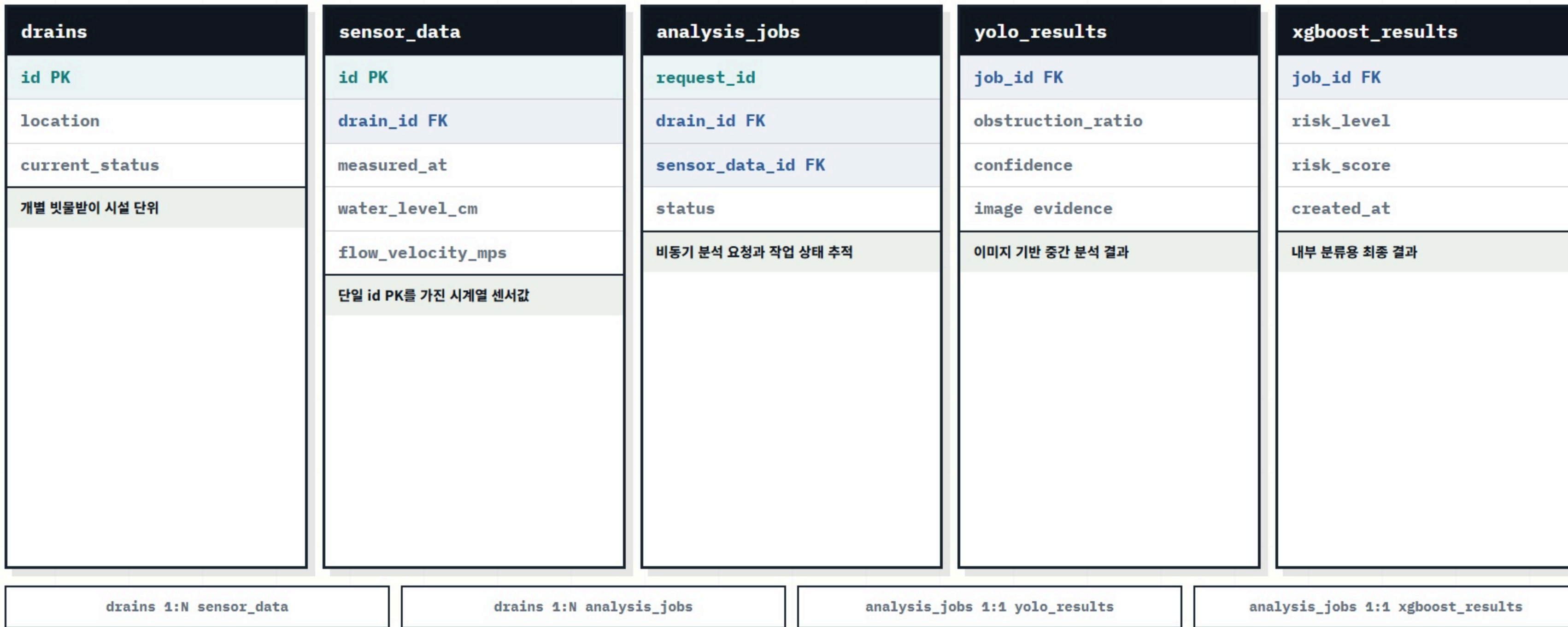
2 센서 시계열: mock sensor data의 수위·유속 추이를 봅니다.

3 위험도: 내부 분류 결과와 상태 라벨을 확인합니다.

4 메타정보: request_id, 시간, 작업 상태를 추적합니다.

DB 설계 / ERD

callback과 WebSocket을 설명하기 전에, 어떤 테이블에 어떤 근거가 저장되는지 먼저 고정합니다.



센서 데이터 시계열 조회 기준

‘sensor_data’의 기본키는 단일 ‘id’이고, 분석 조회는 ‘drain_id + measured_at’ 조건으로 최신 센서 row를 찾습니다.

drain_id 선택 →

measured_at 정렬

latest row 조회 →

AnalysisJob 연결

복합 기본키로 설명하지 않습니다. 필요한 경우 조회 성능을 위해 ‘drain_id’, ‘measured_at’ 중심의 index 전략을 별도로 설명합니다.

기준	역할	발표 포인트
id	sensor_data 단일 primary key	row 식별 기준
drain_id	시설별 센서값 묶음	어느 빗물받이의 데이터인지 연결
measured_at	시계열 정렬 기준	분석 시점에 가까운 최신값 조회

비동기 분석 요청

분석 요청은 화면 응답과 분리하고, AnalysisJob 상태로 어디까지 처리됐는지 추적합니다.



요청 설계 포인트

- scheduler 또는 운영 트리거가 분석 작업을 만듭니다.
- Backend는 request_id와 작업 상태를 기준으로 AI server에 요청합니다.
- 분석 실패는 화면 오류가 아니라 job 상태와 로그로 추적합니다.

callback 저장

callback은 AI server의 결과를 Backend가 검증한 뒤 PostgreSQL 저장과 job status 갱신으로 마무리하는 지점입니다.

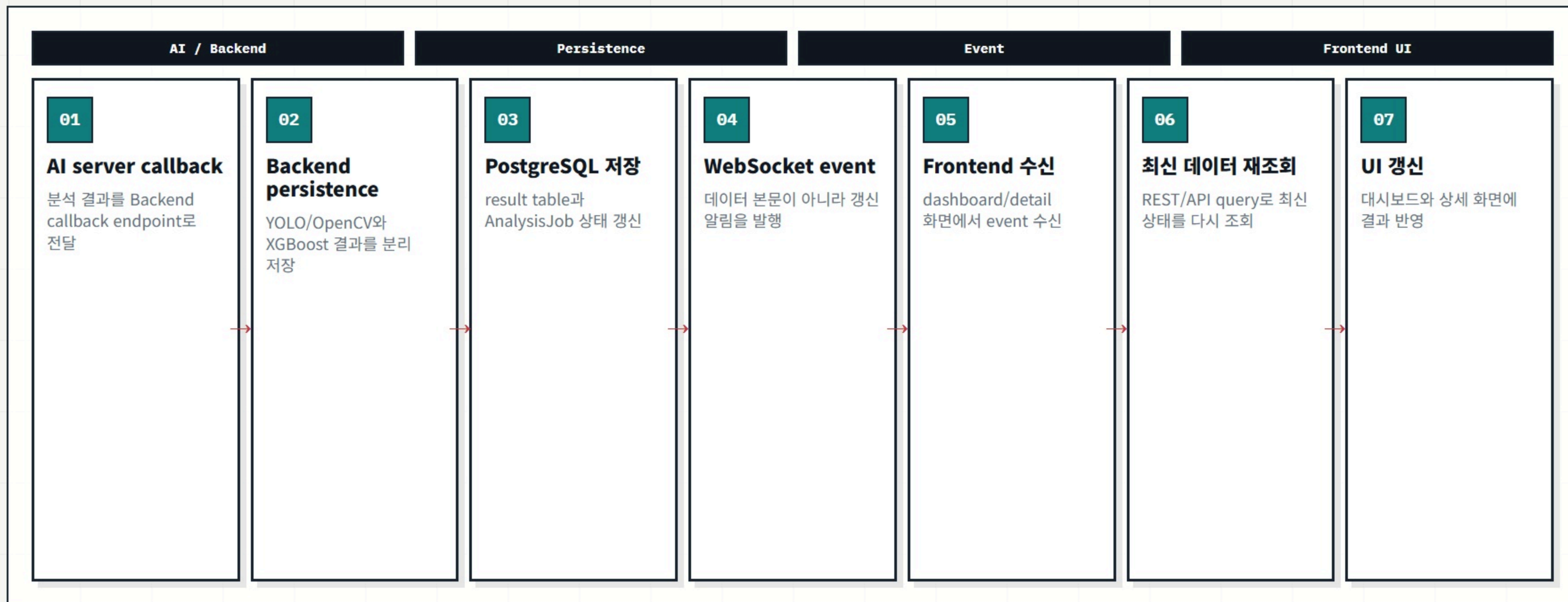
단 계	처 리 내 용	실 패 기 준
callback endpoint	request_id와 결과 payload 수신	잘못된 request_id 또는 중복 요청 확인
PostgreSQL 저장	YOLO/OpenCV, XGBoost 결과 분리 저장	저장 실패 시 job failed 후보
job status 갱신	done 또는 failed로 분석 작업 종료	idempotency 기준으로 중복 반영 방지



callback 실패와 중복 요청은 complete monitoring 구현이 아니라 운영 점검 대상으로 제시합니다.

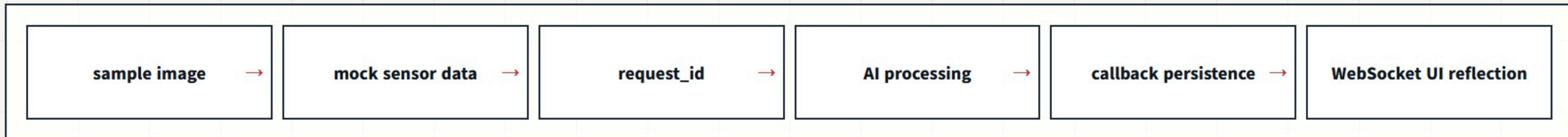
WebSocket 갱신

WebSocket은 전체 데이터를 직접 운반하는 통로가 아니라, 최신 데이터 재조회를 시작하는 화면 갱신 trigger입니다.



전체 서비스 흐름 요약

하나의 request_id가 sample image 입력부터 DB 저장과 UI 반영까지 이동하는 흐름으로 보면 전체 구조가 정리됩니다.



분석 분리

화면 응답과 AI 처리 시간을 분리합니다.

추적성

요청, 센서 row, 분석 결과를 DB 기준으로 연결합니다.

비동기 처리

pending, processing, done, failed 상태를 관리합니다.

화면 반영

event 이후 최신 데이터를 다시 조회합니다.

시스템 아키텍처

현재 구현 구조와 향후 확장 범위를 시각적으로 분리해 과장 없이 설명합니다.

Browser

관리자 대시보드와 상세 화면 진입점

Nginx

Frontend, Backend API, WebSocket routing entry

Frontend

Next.js dashboard/detail UI와 event 수신

Backend

FastAPI, AnalysisJob, callback 저장

PostgreSQL

sensor_data, analysis_jobs, yolo/xgboost results 저장

AI server

YOLO, OpenCV, XGBoost 분석 처리

Docker / Jenkins

서비스 실행 단위와 build/deploy 흐름 구성

RTSP / MQTT / Alert

현재 구현이 아니라 향후 확장 범위

□ 현재 구현 구조 ▤ 향후 확장 범위

Docker / Nginx / Jenkins

배포 관점에서는 GitHub 변경이 Jenkins, Docker build, Compose 실행, Nginx routing으로 이어집니다.



운영 완료 시스템이 아니라, 발표 시점의 build/deploy 흐름 구성으로 설명합니다.

구성	역할	검증 포인트
Docker	서비스별 실행 환경 구성	컨테이너 실행과 환경 변수
Nginx	Frontend, API, WebSocket routing	경로와 WebSocket proxy
Jenkins	GitHub 변경 이후 build/deploy 흐름	빌드 로그와 배포 상태

PostgreSQL 저장 구조

ERD가 아니라 저장 책임 관점에서 보면 시설, 센서 row, 분석 job, AI result, 화면 상태가 차례로 이어집니다.

facility →

sensor row →

analysis job →

AI result →

screen state

저장 목적

시설 단위 상태, 센서 시계열, 분석 결과를 재조회 가능하게 남깁니다.

조회 기준

drain_id, measured_at, request_id로 화면과 분석 결과를 연결합니다.

화면 반영 기준

저장된 최신 결과가 dashboard/detail 상태 표시의 근거가 됩니다.

결과 분리

YOLO/OpenCV 중간 결과와 XGBoost 최종 결과를 분리해 설명합니다.

운영 모니터링 고려

완성된 production monitoring이라고 주장하지 않고, 운영 시 확인해야 할 대상과 대응 방향을 정리합니다.

감지 대상	확인 방법	대응	운영 목적
Backend health	health check, job 상태	API 재시작 또는 로그 확인	요청 처리 지속성
PostgreSQL	connection, migration, storage	연결과 스키마 확인	결과 저장 안정성
AI 실패 로그	timeout, failed job	재시도 또는 판단불가 표시	분석 실패 추적
WebSocket / callback	연결 상태, 중복 request	재연결, idempotency 확인	화면 갱신 신뢰성

향후 RTSP, MQTT, 알림 확장

현재 구현은 sample image와 mock sensor data 기반이며, 실제 CCTV·센서·알림 자동화는 다음 확장 단계로 분리합니다.

현재 구현

sample / mock

sample image, mock sensor data, async analysis, result persistence, WebSocket UI reflection을 시연합니다.

다음 확장

RTSP / MQTT

RTSP CCTV 입력과 MQTT sensor 연동을 실제 입력 채널로 검토합니다.

기대 효과

Alert / Report

상태 변화 알림과 리포트 자동화를 운영 확장 방향으로 제시합니다.

우선순위: 현재 흐름 안정화 -> 실제 입력 연동 -> 알림과 리포트 자동화.

시연 시나리오

발표자는 sample image와 mock sensor data가 들어와 작업 생성, 결과 저장, 화면 갱신으로 이어지는 장면을 보여줍니다.



입력 준비 화면: 시연용 sample image와 mock sensor data가 함께 들어간다고 설명

1

입력 준비

sample drain image와 mock sensor data를 준비합니다.

2

작업 생성

Backend가 AnalysisJob을 만들고 AI server 요청으로 이어집니다.

3

결과 저장

callback 결과가 PostgreSQL에 저장되고 job 상태가 갱신됩니다.

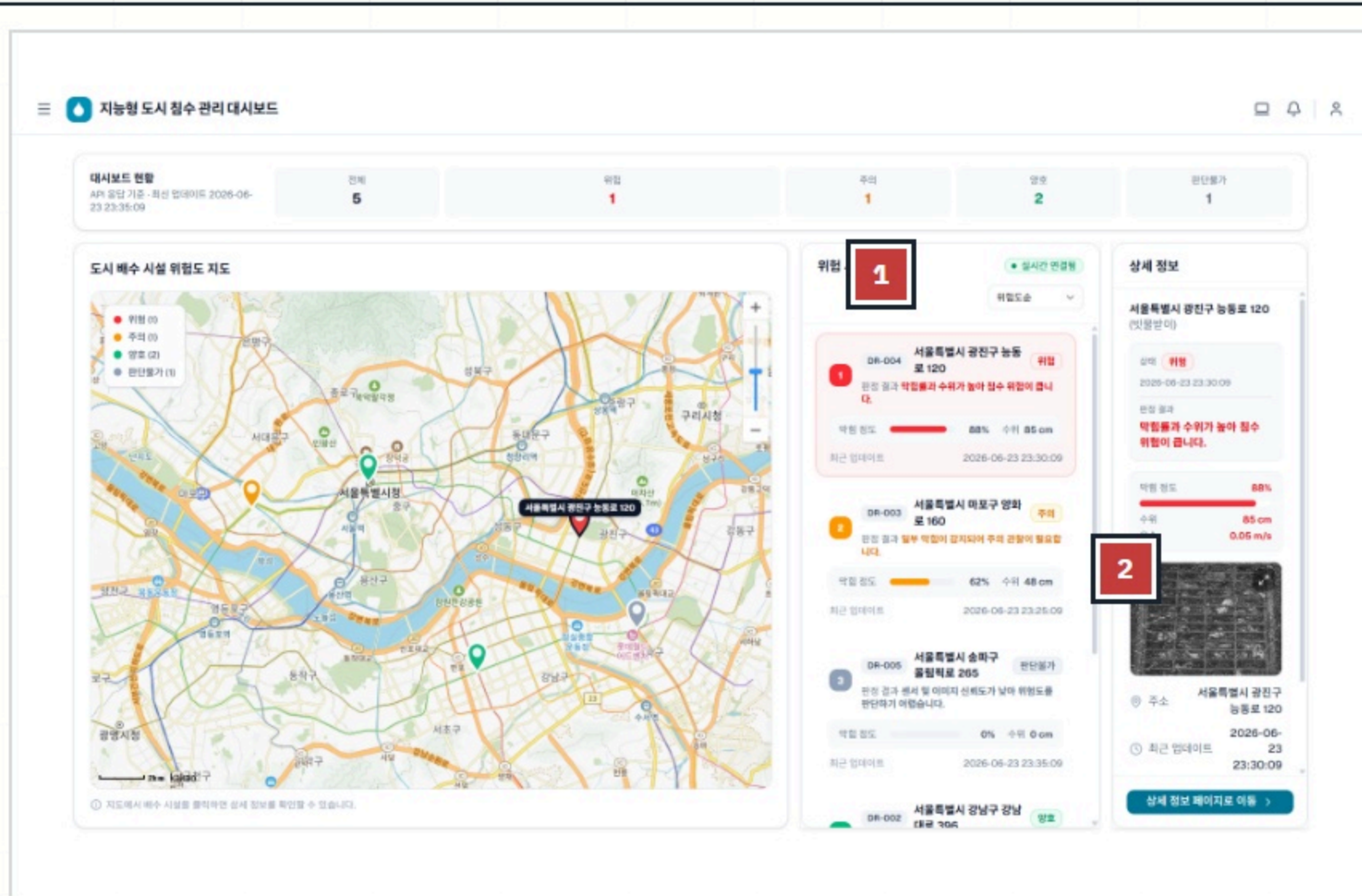
4

화면 갱신

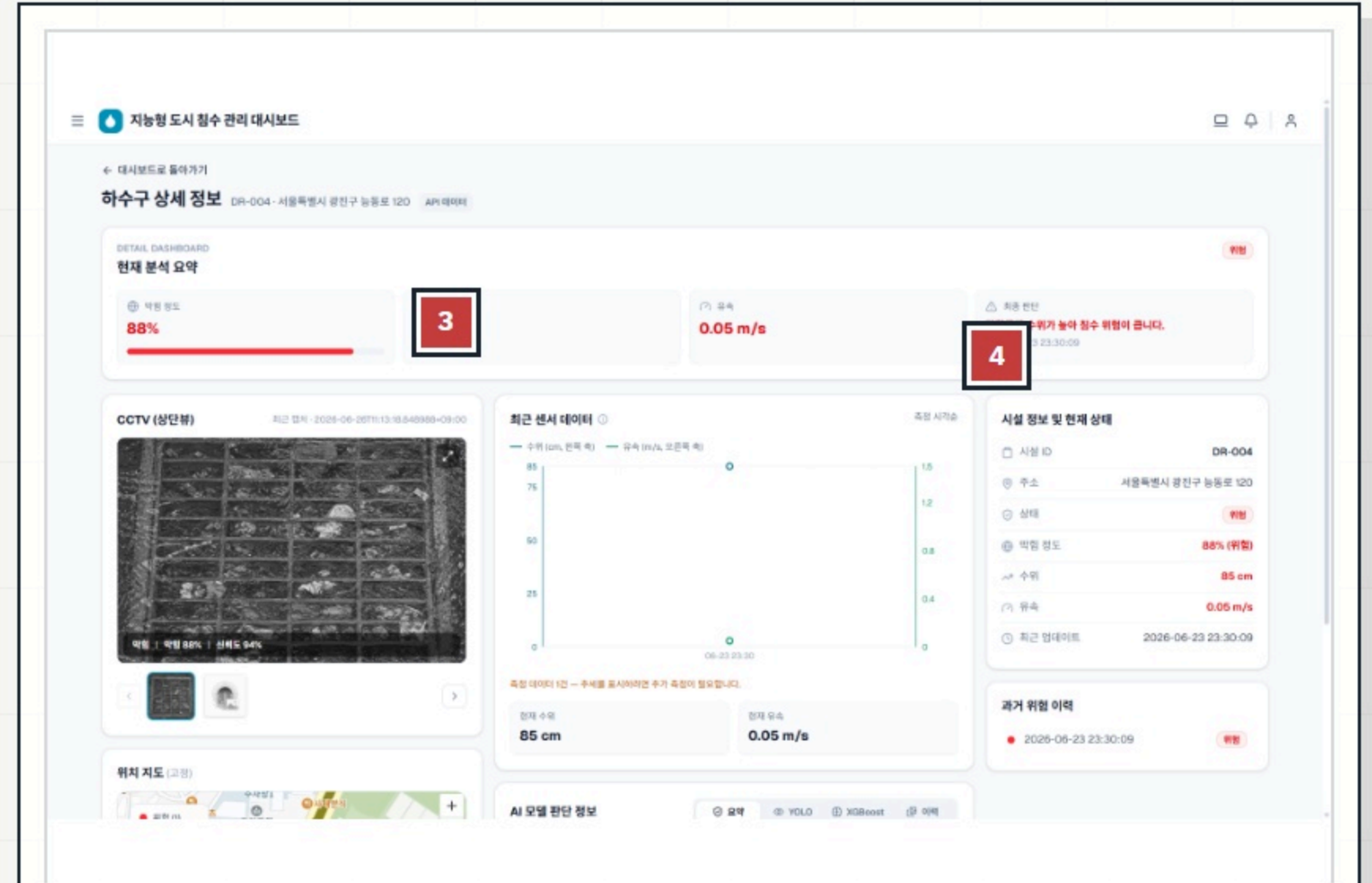
WebSocket event 이후 dashboard/detail UI가 최신 데이터를 반영합니다.

시연 흐름

대시보드에서 위험 시설을 고르고, 상세 화면에서 이미지·센서·AI 근거를 확인하는 순서로 시연합니다.



1. 위험 시설 확인 -> 2. 상세 화면 진입



3. 이미지·센서 근거 확인 -> 4. AI 결과와 이력 확인

기대 효과 / Q&A

질문 받겠습니다

시설별 상태 확인, 위험 시설 우선 확인, 판단 근거 저장과
화면 반영 흐름을 제시했습니다.

정리

Q&A

기억할 핵심 1

현재 구현 범위는 sample images, mock sensor data, async analysis, result persistence, WebSocket UI reflection입니다.

기억할 핵심 2

RTSP, MQTT, 알림, 리포트는 현재 구현이 아니라 향후 확장 범위입니다.



우수주의보

GitHub 또는 Demo 확인용 QR 코드